LISTA DE ACTIVIDADES

Nombre del Proyecto	Rise of the Lost Colors	**Código del Proyecto**	RLC-2026	**Fecha de Elaboración**	27/02/2026

ID	DENOMINACIÓN	DESCRIPCIÓN	ID PAQUETE DE TRABAJO
A1	Redactar Game Design Document	Elaborar el documento completo de diseño del videojuego, incluyendo historia, mecánicas, progresión, enemigos, sistema de poderes, interfaz y estilo artístico, asegurando coherencia global.	1.1.1
A2	Revisión y validación final del documento	Revisar el GDD en profundidad, corregir inconsistencias, validar decisiones técnicas y aprobar la versión final como referencia oficial del desarrollo.	1.1.1
A3	Redactar plan de dirección del proyecto	Elaborar la planificación general del proyecto, incluyendo alcance, objetivos, cronograma, recursos, presupuesto, gestión de riesgos y criterios de seguimiento y control. Su finalidad es establecer el marco de gestión que guiará la ejecución y asegurará el cumplimiento de los objetivos del proyecto.	1.1.2
A4	Redactar plan de gestión de requisitos	Redactar un documento que define los procesos para identificar, analizar, documentar, priorizar, validar y controlar los requisitos del proyecto.	1.1.3
A5	Redacción del registro de requisitos	Análisis exhaustivo de las necesidades de los usuarios y transformación en un listado formal de requisitos funcionales y no funcionales, documentados con criterios de aceptación claros, priorizados mediante técnicas como MoSCoW y organizados en un registro trazable que servirá como referencia durante todo el ciclo de vida del proyecto.	1.1.3
A6	Elaboración de la matriz de requisitos y seguimiento	Creación y mantenimiento de una matriz de trazabilidad bidireccional que vincula sistemáticamente cada requisito con sus entregables correspondientes, características del producto, fuentes de origen y criterios de validación, asegurando la cobertura completa y el cumplimiento verificable de todos los requisitos.	1.1.3
A7	Redactar enunciado del alcance	Redactar un documento que detalla el trabajo que debe realizarse para entregar los productos del proyecto, incluyendo los objetivos, entregables principales, criterios de aceptación y exclusiones. Sirve de base para definir el alcance detallado.	1.1.4

A8	Redactar plan de gestión del alcance	Redactar un documento que establece los procedimientos para definir, validar y controlar el alcance del proyecto.	1.1.4
A9	Redactar plan de gestión de costes	Realizar la estimación, planificación y control del presupuesto del proyecto, incluyendo la identificación de recursos necesarios y el seguimiento de los costes para asegurar que el proyecto se mantenga dentro del presupuesto establecido.	1.1.5
A10	Redactar plan de gestión del cronograma	Elaborar la planificación temporal del proyecto, definiendo actividades, secuencias, duraciones e hitos, así como en el seguimiento del avance para garantizar el cumplimiento de los plazos previstos.	1.1.6
A11	Redactar plan de gestión de riesgos	Redactar un documento en el que se identifiquen los procesos para la detección, análisis, planificación de respuesta y monitoreo de riesgos. Su objetivo es minimizar las amenazas y aprovechar las oportunidades a lo largo del ciclo de vida del proyecto.	1.1.7
A12	Redacción del registro de riesgos	Identificación proactiva, análisis cualitativo y documentación estructurada de todos los eventos inciertos que podrían afectar positiva o negativamente a los objetivos del proyecto, describiendo causas raíz, impactos potenciales cuantificados, probabilidades de ocurrencia y estableciendo un sistema de monitoreo continuo	1.1.7
A13	Redactar plan de gestión de calidad	Definir los estándares y criterios de calidad del proyecto, así como a establecer actividades de aseguramiento y control que permitan verificar que los entregables cumplen los requisitos definidos.	1.1.8
A14	Redacción del registro de incidencias	Documentar todas las incidencias ocurridas durante la ejecución del proyecto, proporcionando una descripción de esta.	1.1.9
A15	Redacción del registro de decisiones	Registrar de forma sistemática todas las decisiones tomadas durante el proyecto, incluyendo su justificación, responsables y fecha, para asegurar la trazabilidad y facilitar el seguimiento.	1.1.9
A16	Redacción del registro de cambios	Documentar todas las solicitudes de cambio en el alcance que se hayan recibido a lo largo de la fase de ejecución del proyecto. Se proporcionará una descripción detallada de la	1.1.9

		solicitud, un análisis de las partes afectas y su impacto en ellas, la fuente de la solicitud y la resolución de la misma (aceptada o denegada).	
A17	Reuniones internas del equipo	Realizar reuniones periódicas entre los miembros del equipo de desarrollo con el objetivo de coordinar tareas, revisar el progreso de las actividades planificadas, identificar posibles bloqueos o problemas técnicos y asegurar una comunicación fluida entre los integrantes, facilitando la toma de decisiones y la correcta distribución del trabajo.	1.1.9
A18	Reuniones con el tutor	Mantener reuniones de seguimiento con el tutor del proyecto para presentar avances del desarrollo, resolver dudas técnicas o metodológicas y recibir orientación sobre el enfoque del trabajo, garantizando que el proyecto se mantenga alineado con los objetivos académicos y los requisitos establecidos.	1.1.9
A19	Sprint Reviews	Realizar sesiones de revisión al final de cada sprint con el objetivo de presentar las funcionalidades implementadas durante el periodo de desarrollo, evaluar el grado de cumplimiento de los objetivos planificados y recoger feedback que permita mejorar el producto y ajustar la planificación de los siguientes ciclos de trabajo.	1.1.9
A20	Definir atributos y habilidades base del jugador	Establecer los valores iniciales que determinarán el rendimiento del personaje durante la partida, como puntos de vida máximos, velocidad de desplazamiento, fuerza de ataque, resistencia al daño y posibles atributos secundarios. Estos valores deben equilibrarse cuidadosamente para garantizar una experiencia progresiva adecuada, evitando que el jugador resulte demasiado débil o excesivamente poderoso en las primeras fases del juego.	2.1.1
A21	Especificar acciones disponibles	Identificar y documentar todas las acciones que el jugador podrá realizar, incluyendo movimientos básicos, acciones ofensivas, defensivas y habilidades especiales. Además, se detallarán las condiciones de activación, restricciones temporales y posibles combinaciones entre acciones, asegurando coherencia mecánica dentro del sistema general del juego.	2.1.1
A22	Definir de interacciones posibles con el entorno	Describir cómo el jugador interactúa con plataformas, enemigos, trampas, objetos coleccionables y elementos dinámicos del escenario. Se deben definir las reglas de colisión, activación de eventos y consecuencias derivadas de cada interacción para evitar comportamientos inconsistentes.	2.1.1

A23	Establecer limitaciones	Determinar las restricciones técnicas que afectan al personaje, como gravedad aplicada, velocidad máxima, altura de salto, fricción en superficies y límites del mapa. Estas limitaciones garantizan una experiencia de juego controlada y coherente con las físicas definidas.	2.1.1
A24	Implementar movimiento lateral	Desarrollar el sistema de desplazamiento horizontal aplicando aceleración progresiva y desaceleración natural, de forma que el movimiento resulte fluido y responsivo. También se gestionarán las colisiones con obstáculos y límites del escenario.	2.1.2
A25	Implementar salto	Programar la mecánica de salto considerando fuerza inicial, efecto de la gravedad y control de altura variable según la duración de la pulsación. Se deben validar correctamente las condiciones de salto desde el suelo.	2.1.2
A26	Implementar doble salto	Añadir una segunda acción de salto ejecutable únicamente mientras el jugador esté en el aire. Esta mecánica deberá incluir validaciones para impedir su uso ilimitado y mantener el equilibrio del diseño.	2.1.2
A27	Implementar dash	Desarrollar un impulso rápido en la dirección indicada que permita esquivar ataques o superar obstáculos. El dash incluirá limitaciones de duración y tiempo de reutilización para evitar abusos.	2.1.2
A28	Implementar ataque cuerpo a cuerpo	Programar la acción ofensiva principal del jugador, sincronizando animación, activación del área de impacto y temporización adecuada para evitar ataques continuos sin restricción.	2.1.3
A29	Implementar deteccion de impactos	Desarrollar el sistema de colisiones que permita detectar cuándo el área de ataque entra en contacto con un enemigo, asegurando precisión en el cálculo del impacto.	2.1.3
A30	Integrar con sistema de daño	Conectar la detección de impacto con la reducción de puntos de vida del enemigo, incluyendo posibles efectos adicionales como retroceso o animación de daño.	2.1.3
A31	Implementar sistema de cambio de poderes	Desarrollar una estructura modular que permita alternar entre diferentes poderes activos, garantizando que solo uno esté operativo a la vez y gestionando correctamente su activación y desactivación.	2.1.4

A32	Implementar poder de velocidad	Programar un aumento temporal de la velocidad del jugador que modifique dinámicamente sus parámetros de movimiento, incluyendo control de duración y posibles efectos visuales.	2.1.4
A33	Implementar poder de fuerza	Aumentar temporalmente el daño infligido por los ataques del jugador, ajustando el cálculo del daño dentro del sistema de combate.	2.1.4
A34	Implementar poder de escudo	Desarrollar un sistema defensivo que absorba o reduzca el daño recibido durante un tiempo limitado, incluyendo indicadores visuales que informen de su activación.	2.1.4
A35	Implementar sistema de salud	Crear la estructura de datos que almacene la vida actual y máxima del jugador, incluyendo funciones para su modificación y consulta desde otros sistemas.	2.1.5
A36	Implementar sistema de daño infligido	Definir cómo el jugador pierde puntos de vida al recibir impactos, asegurando correcta comunicación con enemigos y trampas.	2.1.5
A37	Implementar invulnerabilidad temporal	Tras recibir daño, se activará un periodo breve durante el cual el jugador no podrá ser herido nuevamente, evitando acumulación instantánea de impactos	2.1.5
A38	Implementar muerte y reaparición	Desarrollar la secuencia de muerte del personaje, incluyendo animación, reinicio del estado y reaparición en el último punto de control activado.	2.1.5
A39	Crear sprite base del jugador	Diseñar el modelo visual principal del personaje jugable respetando el estilo artístico global del videojuego, definiendo paleta de colores y nivel de detalle, así como su correcta alineación con la hitbox para garantizar coherencia entre representación gráfica y sistema de colisiones, optimizando además el formato para su integración eficiente en el motor de juego.	2.1.6
A40	Animar estado de reposo	Crear una animación cíclica fluida que se reproduzca cuando el jugador no realiza acciones, incorporando movimientos sutiles que aporten dinamismo y personalidad al personaje sin alterar su posición real ni su hitbox, ajustando número de fotogramas y velocidad de reproducción para mantener coherencia con el ritmo general del juego.	2.1.6
A41	Animar movimientos	Desarrollar la secuencia animada asociada al desplazamiento lateral del personaje, asegurando que transmita sensación de fluidez, peso y velocidad, sincronizando los	2.1.6

		fotogramas con la velocidad real del sistema de movimiento y garantizando transiciones suaves entre estados como reposo y desplazamiento.	
A42	Animar ataques	Crear la animación correspondiente a la acción ofensiva del jugador, definiendo claramente las fases de preparación, impacto y recuperación, sincronizando el momento exacto de activación de la hitbox con el fotograma de impacto y asegurando que la animación respete las restricciones mecánicas establecidas en el sistema de combate.	2.1.6
A43	Animar recepción de daño	Desarrollar la secuencia visual que representa la recepción de daño por parte del jugador, incorporando efectos como retroceso, cambios temporales de color o parpadeo para indicar vulnerabilidad, y sincronizándola correctamente con el sistema de invulnerabilidad temporal y la lógica de interrupción de acciones.	2.1.6
A44	Crear variantes según poderes	Diseñar modificaciones visuales del personaje que reflejen de forma clara e inmediata el poder actualmente activo, mediante cambios de color, efectos gráficos o elementos adicionales, asegurando coherencia estética, correcta integración con las animaciones existentes y sincronización precisa con la activación lógica del poder.	2.1.6
A45	Diseñar enemigos de cada nivel	Definir las distintas tipologías de enemigos que aparecerán en cada nivel del videojuego, estableciendo sus características diferenciadoras en términos de rol (ofensivo, defensivo, móvil, estático), nivel de amenaza y coherencia con la ambientación, asegurando que su diseño contribuya a la progresión de dificultad y a la variedad jugable a lo largo de la experiencia.	2.2.1
A46	Especificar comportamientos	Especificar los patrones de comportamiento fundamentales de cada tipo de enemigo, tales como patrullaje, persecución del jugador, ataque a corta o larga distancia, espera activa o activación por proximidad, detallando las condiciones de transición entre estados para garantizar coherencia dentro del sistema de inteligencia artificial.	2.2.1
A47	Definir estadísticas base	Determinar los valores base de atributos como puntos de vida, daño infligido, velocidad de desplazamiento, rango de detección y frecuencia de ataque, equilibrándolos en función del momento de aparición dentro del juego y del nivel de progresión del jugador, con el objetivo de mantener un desafío constante y ajustado.	2.2.1

A48	Implementar detección del jugador	Desarrollar el sistema que permita a los enemigos identificar la presencia del jugador dentro de un rango determinado, utilizando mecanismos como detección por distancia, línea de visión o colisión con áreas de activación, asegurando que la activación del comportamiento ofensivo se produzca de manera coherente y optimizada en términos de rendimiento.	2.2.2
A49	Implementar movimiento	Implementar el desplazamiento automático de los enemigos según los patrones definidos en el diseño conceptual, como patrullaje entre puntos, seguimiento del jugador o movimiento fijo en un área concreta, garantizando una integración correcta con el sistema de físicas y colisiones del entorno.	2.2.2
A50	Implementar ataques	Programar la lógica de activación del ataque cuando el jugador se encuentre dentro del rango efectivo, definiendo temporización, frecuencia de ataque y posibles tiempos de recuperación, asegurando que el comportamiento ofensivo sea predecible pero desafiante.	2.2.2
A51	Implementar gestión de estados	Implementar una máquina de estados que controle las distintas fases de comportamiento del enemigo (reposo, patrullaje, persecución, ataque, daño y muerte), definiendo claramente las condiciones de transición entre estados para garantizar coherencia lógica y evitar comportamientos erráticos.	2.2.2
A52	Implementar sistemas de spawn de enemigos	Configurar los puntos y condiciones de aparición de los enemigos dentro del nivel, definiendo si su activación es automática, dependiente de la proximidad del jugador o ligada a eventos específicos, asegurando un control adecuado del ritmo de juego y del rendimiento del sistema.	2.2.3
A53	Implementar despawn automático de enemigos	Desarrollar el sistema que gestione la desaparición de enemigos cuando son derrotados o cuando abandonan el área activa del nivel, liberando recursos del sistema y evitando acumulación innecesaria de entidades en memoria.	2.2.3
A54	Implementar estadísticas de enemigos de nivel 1	Configurar los puntos de vida y daño de los enemigos del primer nivel para ofrecer un desafío moderado acorde a las capacidades básicas del jugador en las primeras fases del juego.	2.2.4

A55	Implementar estadísticas de enemigos de nivel 2	Configurar los puntos de vida y daño de los enemigos del segundo nivel para ofrecer un desafío moderado acorde a las capacidades del jugador.	2.2.4
A56	Implementar estadísticas de enemigos de nivel 3	Configurar los puntos de vida y daño de los enemigos del tercer nivel para ofrecer un desafío moderado acorde a las capacidades del jugador.	2.2.4
A57	Implementar estadísticas de enemigos de nivel final	Configurar los puntos de vida y daño de los enemigos del nivel final para ofrecer un desafío moderado acorde a las capacidades del jugador.	2.2.4
A58	Crear sprites y animar enemigos de nivel 1	Diseñar la representación visual de los enemigos correspondientes al primer nivel, definiendo su silueta, proporciones, paleta de colores y nivel de detalle de forma coherente con la ambientación inicial del juego, asegurando además que su diseño permita una identificación clara de su rol y nivel de amenaza.	2.2.5
A59	Crear sprites y animar enemigos de nivel 2	Diseñar la representación visual de los enemigos correspondientes al segundo nivel, definiendo su silueta, proporciones, paleta de colores y nivel de detalle de forma coherente con la ambientación inicial del juego, asegurando además que su diseño permita una identificación clara de su rol y nivel de amenaza.	2.2.5
A60	Crear sprites y animar enemigos de nivel 3	Diseñar la representación visual de los enemigos correspondientes al tercer nivel, definiendo su silueta, proporciones, paleta de colores y nivel de detalle de forma coherente con la ambientación inicial del juego, asegurando además que su diseño permita una identificación clara de su rol y nivel de amenaza.	2.2.5
A61	Crear sprites y animar enemigos de nivel final	Diseñar la representación visual de los enemigos correspondientes al nivel final, definiendo su silueta, proporciones, paleta de colores y nivel de detalle de forma coherente con la ambientación inicial del juego, asegurando además que su diseño permita una identificación clara de su rol y nivel de amenaza.	2.2.5
A62	Definir patrones de ataque base de Umbra	Definir el comportamiento inicial del jefe evolutivo, estableciendo sus patrones de ataque, ritmo de combate, movimientos característicos y mecánicas diferenciadoras respecto a los enemigos básicos, asegurando que el enfrentamiento resulte único y estratégico dentro de la experiencia de juego.	2.3.1

A63	Implementar recompensas tras derrota de Umbra	Establecer las mejoras, habilidades desbloqueadas o progresos narrativos que el jugador obtendrá al derrotar al jefe, garantizando que la recompensa sea proporcional al nivel de dificultad y contribuya significativamente a la progresión del juego.	2.3.1
A64	Desarrollar modelo de IA para aprendizaje	Desarrollar la lógica inicial de comportamiento del jefe, programando sus movimientos, selección de ataques, tiempos de espera y condiciones de activación, integrando una máquina de estados que permita gestionar correctamente las distintas acciones durante el combate y garantizando coherencia con el sistema general de físicas y colisiones del juego.	2.3.2
A65	Implementar modelo en la toma de decisiones	Diseñar e implementar un sistema de comportamiento dinámico que permita a Umbra modificar su estrategia en función del desempeño del jugador, analizando variables como daño recibido, tiempo de combate o patrones repetitivos del jugador, con el objetivo de aumentar la complejidad del enfrentamiento y evitar estrategias excesivamente predecibles.	2.3.2
A66	Implementar modificación de atributos	Implementar un mecanismo que incremente gradualmente la capacidad ofensiva del jefe a medida que avanza el combate o se activan nuevas fases, ajustando los valores de daño y frecuencia de ataque para intensificar la presión sobre el jugador.	2.3.3
A67	Implementar cambio de comportamiento	Desarrollar un sistema que modifique los atributos defensivos de Umbra, como puntos de vida o reducción de daño recibido, en función de su estado evolutivo, asegurando que cada fase represente un desafío mayor y diferenciado.	2.3.3
A68	Implementar condiciones de activación	Programar transiciones entre distintas fases del combate que alteren los patrones de ataque, velocidad de movimiento o habilidades disponibles del jefe, definiendo claramente las condiciones de activación de cada fase y garantizando una transición fluida tanto a nivel lógico como visual.	2.3.3
A69	Crear Sprite base de Umbra	Crear la representación visual inicial del jefe evolutivo, definiendo su identidad gráfica, proporciones, silueta y estilo acorde a su rol central dentro del juego, asegurando que su diseño transmita claramente su importancia y nivel de amenaza.	2.3.4

A70	Crear variantes evolutivas	Desarrollar una versión visual alternativa que represente la transformación del jefe durante el combate, incorporando cambios perceptibles en estructura, tamaño, colores o efectos gráficos que refuercen la sensación de evolución y aumento de poder.	2.3.4
A71	Animar ataques, daño y derrota	Crear las animaciones asociadas a todas sus acciones relevantes (movimiento, ataques, recepción de daño, transición de fase y derrota), sincronizándolas con la lógica de la IA y los momentos clave del combate para garantizar coherencia visual, claridad en el feedback al jugador y correcta integración técnica en el motor de juego.	2.3.4
A72	Definir patrones del Guardian de Hielo	Diseñar los patrones de comportamiento del Guardián de Hielo estableciendo su lógica de ataque, ritmo de combate y mecánicas específicas, definiendo las secuencias de habilidades que generen un enfrentamiento estratégico centrado en la gestión del movimiento del jugador.	2.4.1
A73	Definir patrones del Coloso de Ceniza	Diseñar los patrones de comportamiento del Coloso de Ceniza estableciendo su lógica de ataque, ritmo de combate y mecánicas específicas, definiendo las secuencias de habilidades que generen un enfrentamiento estratégico centrado en la gestión del movimiento del jugador.	2.4.1
A74	Definir patrones de la Tempestad Dorada	Diseñar los patrones de comportamiento de la Tempestad Dorada estableciendo su lógica de ataque, ritmo de combate y mecánicas específicas, definiendo las secuencias de habilidades que generen un enfrentamiento estratégico centrado en la gestión del movimiento del jugador.	2.4.1
A75	Definir patrones del Vacío	Diseñar los patrones de comportamiento del Vacío estableciendo su lógica de ataque, ritmo de combate y mecánicas específicas, definiendo las secuencias de habilidades que generen un enfrentamiento estratégico centrado en la gestión del movimiento del jugador.	2.4.1
A76	Implementar al Guardian de Hielo	Desarrollar la implementación técnica completa del Guardián de Hielo integrando su máquina de estados, ataques definidos, detección del jugador, gestión de colisiones y sincronización con animaciones, garantizando estabilidad del sistema y coherencia entre diseño conceptual y comportamiento real en el motor de juego.	2.4.2

A77	Implementar al Coloso de Ceniza	Desarrollar la implementación técnica completa del Coloso de Ceniza integrando su máquina de estados, ataques definidos, detección del jugador, gestión de colisiones y sincronización con animaciones, garantizando estabilidad del sistema y coherencia entre diseño conceptual y comportamiento real en el motor de juego.	2.4.2
A78	Implementar a la Tempestad Dorada	Desarrollar la implementación técnica completa de la Tempestad Dorada integrando su máquina de estados, ataques definidos, detección del jugador, gestión de colisiones y sincronización con animaciones, garantizando estabilidad del sistema y coherencia entre diseño conceptual y comportamiento real en el motor de juego.	2.4.2
A79	Implementar al Vacío	Desarrollar la implementación técnica completa del Vacío integrando su máquina de estados, ataques definidos, detección del jugador, gestión de colisiones y sincronización con animaciones, garantizando estabilidad del sistema y coherencia entre diseño conceptual y comportamiento real en el motor de juego.	2.4.2
A80	Implementar zonas de vulnerabilidad del Guardian de Hielo	Desarrollar el sistema de hitboxes específicas que determinen momentos o áreas concretas donde el Guardián de Hielo pueda recibir daño aumentado, definiendo condiciones de activación vinculadas a animaciones o finalización de ataques para fomentar el combate estratégico.	2.4.3
A81	Implementar zonas de vulnerabilidad del Coloso de Ceniza	Desarrollar el sistema de hitboxes específicas que determinen momentos o áreas concretas donde el Colos de Ceniza pueda recibir daño aumentado, definiendo condiciones de activación vinculadas a animaciones o finalización de ataques para fomentar el combate estratégico.	2.4.3
A82	Implementar zonas de vulnerabilidad de la Tempestad Dorada	Desarrollar el sistema de hitboxes específicas que determinen momentos o áreas concretas donde la Tempestad Dorada pueda recibir daño aumentado, definiendo condiciones de activación vinculadas a animaciones o finalización de ataques para fomentar el combate estratégico.	2.4.3
A83	Crear sprite y animaciones del Guardian de Hielo	Diseñar el apartado visual completo del Guardián de Hielo incluyendo sprite base, animaciones de movimiento, ataque y derrota, garantizando coherencia con su temática helada y correcta integración técnica con el sistema de estados.	2.4.4

A84	Crear sprite y animaciones del Coloso de Ceniza	Diseñar el apartado visual completo del Coloso de Ceniza incluyendo sprite base, animaciones de movimiento, ataque y derrota, garantizando coherencia con su temática de fuego y correcta integración técnica con el sistema de estados.	2.4.4
A85	Crear sprite y animaciones de la Tempestad Dorada	Diseñar el apartado visual completo de la Tempestad Dorada incluyendo sprite base, animaciones de movimiento, ataque y derrota, garantizando coherencia con su temática eléctrica y correcta integración técnica con el sistema de estados.	2.4.4
A86	Crear sprite y animaciones del Vacío	Diseñar el apartado visual completo del Vacío incluyendo sprite base, animaciones de movimiento, ataque y derrota, garantizando coherencia con su temática oscura y correcta integración técnica con el sistema de estados.	2.4.4
A87	Diseñar layout del nivel tutorial	Definir la estructura espacial del nivel tutorial estableciendo distribución de plataformas, zonas seguras, obstáculos y primeros enemigos, con el objetivo de introducir progresivamente las mecánicas básicas del juego, guiando al jugador mediante un diseño intuitivo que favorezca el aprendizaje sin sobrecargar la experiencia inicial.	2.5.1
A88	Diseñar layout del nivel 1	Diseñar la disposición general del primer nivel formal del juego, equilibrando exploración, combate y secciones de dificultad moderada, asegurando coherencia con la progresión tras el tutorial e introduciendo nuevos elementos sin romper la curva de aprendizaje.	2.5.1
A89	Diseñar layout del nivel 2	Planificar la estructura del segundo nivel incrementando la complejidad del entorno mediante mayor densidad de enemigos, variaciones verticales y obstáculos adicionales, buscando una experiencia más desafiante que exija dominio de las mecánicas previamente aprendidas.	2.5.1
A90	Diseñar layout del nivel 3	Definir un diseño espacial más exigente que combine plataformas complejas, trampas y zonas de combate estratégicas, integrando rutas alternativas o secciones de mayor precisión para aumentar la intensidad y variedad jugable.	2.5.1
A91	Diseñar layout del nivel final	Diseñar la estructura del nivel final del juego asegurando una progresión hacia el enfrentamiento final, incorporando secciones de alta dificultad, preparación previa al jefe y una disposición espacial que genere tensión.	2.5.1

A92	Implementar tilemap del nivel tutorial	Construir el mapa jugable del tutorial utilizando el sistema de tiles del motor, ajustando colisiones, límites y capas visuales para asegurar estabilidad técnica y fidelidad respecto al diseño conceptual.	2.5.2
A93	Implementar tilemap del nivel 1	Construir el mapa jugable del nivel 1 utilizando el sistema de tiles del motor, ajustando colisiones, límites y capas visuales para asegurar estabilidad técnica y fidelidad respecto al diseño conceptual.	2.5.2
A94	Implementar tilemap del nivel 2	Construir el mapa jugable del nivel 2 utilizando el sistema de tiles del motor, ajustando colisiones, límites y capas visuales para asegurar estabilidad técnica y fidelidad respecto al diseño conceptual.	2.5.2
A95	Implementar tilemap del nivel 3	Construir el mapa jugable del nivel 3 utilizando el sistema de tiles del motor, ajustando colisiones, límites y capas visuales para asegurar estabilidad técnica y fidelidad respecto al diseño conceptual.	2.5.2
A96	Implementar tilemap del nivel final	Construir el mapa jugable del nivel final utilizando el sistema de tiles del motor, ajustando colisiones, límites y capas visuales para asegurar estabilidad técnica y fidelidad respecto al diseño conceptual.	2.5.2
A97	Colocar enemigos del nivel 1	Distribuir estratégicamente los enemigos del primer nivel dentro del escenario, considerando ritmo de aparición, dificultad progresiva y equilibrio entre exploración y combate.	2.5.3
A98	Colocar enemigos del nivel 2	Distribuir estratégicamente los enemigos del segundo nivel dentro del escenario, considerando ritmo de aparición, dificultad progresiva y equilibrio entre exploración y combate.	2.5.3
A99	Colocar enemigos del nivel 3	Distribuir estratégicamente los enemigos del tercer nivel dentro del escenario, considerando ritmo de aparición, dificultad progresiva y equilibrio entre exploración y combate.	2.5.3
A100	Colocar enemigos del nivel final	Distribuir estratégicamente los enemigos del nivel final dentro del escenario, considerando ritmo de aparición, dificultad progresiva y equilibrio entre exploración y combate.	2.5.3

A101	Configurar triggers de jefes	Implementar y configurar los eventos de activación que inician los combates contra jefes, definiendo condiciones como entrada a zonas específicas, cierre de puertas o activación de cinemáticas, garantizando sincronización con sistemas de audio y animación.	2.5.3
A102	Implementar trampas	Desarrollar elementos interactivos dañinos como pinchos, proyectiles automáticos o mecanismos móviles, configurando su activación, temporización y detección de colisiones con el jugador.	2.5.4
A103	Implementar plataformas especiales	Programar plataformas con comportamientos diferenciados como movimiento automático, desaparición temporal o activación por presión, integrándolas correctamente con el sistema de físicas.	2.5.4
A104	Implementar zonas de muerte instantánea	Configurar áreas del escenario que provoquen la eliminación inmediata del jugador al entrar en contacto, gestionando reinicios de posición y penalizaciones correspondientes.	2.5.4
A105	Diseñar fondo y efectos del nivel tutorial	Crear el apartado visual de fondo del tutorial incluyendo capas decorativas, efectos ambientales sutiles y coherencia cromática que no distraiga del aprendizaje de mecánicas básicas.	2.5.5
A106	Diseñar fondo y efectos del nivel 1	Desarrollar el diseño visual del nivel 1 incorporando fondos detallados y efectos ambientales acordes a su temática, reforzando la inmersión sin afectar la legibilidad del entorno jugable.	2.5.5
A107	Diseñar fondo y efectos del nivel 2	Desarrollar el diseño visual del nivel 2 incorporando fondos detallados y efectos ambientales acordes a su temática, reforzando la inmersión sin afectar la legibilidad del entorno jugable.	2.5.5
A108	Diseñar fondo y efectos del nivel 3	Desarrollar el diseño visual del nivel 3 incorporando fondos detallados y efectos ambientales acordes a su temática, reforzando la inmersión sin afectar la legibilidad del entorno jugable.	2.5.5
A109	Diseñar fondo y efectos del nivel final	Desarrollar el diseño visual del nivel final incorporando fondos detallados y efectos ambientales acordes a su temática, reforzando la inmersión sin afectar la legibilidad del entorno jugable.	2.5.5
A110	Implementar indicadores de salud	Desarrollar el sistema visual que represente en tiempo real el estado de salud del jugador mediante iconos, asegurando su correcta actualización ante la recepción de daño o	3.1.1

		recuperación, así como su integración coherente dentro del HUD sin interferir con la visibilidad del entorno jugable.	
A111	Implementar indicador de poderes	Programar el elemento de interfaz que muestre la disponibilidad, carga o tiempo de reutilización de habilidades especiales, garantizando que el jugador disponga de información clara y precisa sobre el estado de sus poderes durante el combate.	3.1.1
A112	Implementar recolección de objetos clave	Desarrollar la lógica que permita al jugador recoger objetos esenciales para la progresión, gestionando detección de colisiones y posibles efectos asociados, como desbloqueo de zonas o activación de eventos.	3.1.1
A113	Diseñar interfaz de inicio	Crear el diseño visual de la pantalla principal del juego definiendo disposición de botones, tipografía, identidad gráfica y elementos decorativos, asegurando coherencia estética con la temática general y claridad en las opciones disponibles para el usuario.	3.1.2
A114	Programar navegación de la pantalla de inicio	Implementar la funcionalidad interactiva de la pantalla principal, programando la transición entre opciones como nueva partida, continuar, ajustes o salir, garantizando una navegación fluida y sin errores de estado.	3.1.2
A115	Diseñar interfaz de pausa	Diseñar la estructura visual del menú de pausa incluyendo opciones de reanudación, ajustes o salida, asegurando una presentación clara y coherente con el resto del sistema de interfaz.	3.1.3
A116	Programar lógica de pausa	Desarrollar el sistema que detiene temporalmente la ejecución del juego, congelando movimientos, físicas y temporizadores activos, manteniendo únicamente la funcionalidad del menú hasta que el jugador decida reanudar la partida.	3.1.3
A117	Programar navegación de menú	Implementar la lógica general que gestione la interacción entre distintas pantallas del sistema de menús, controlando estados, transiciones y retorno a pantallas anteriores sin pérdida de información ni inconsistencias.	3.1.3
A118	Diseñar interfaz de ajustes	Crear el diseño visual del menú de configuración incluyendo controles para sonido, resolución, controles u otras opciones, asegurando una distribución clara y accesible que facilite la modificación de parámetros.	3.1.4

A119	Programar lógica de las configuraciones	Desarrollar el sistema que permita aplicar, guardar y recuperar los ajustes definidos por el usuario, garantizando persistencia entre sesiones y correcta aplicación de cambios en tiempo real cuando sea posible.	3.1.4
A120	Diseñar pantalla de carga	Diseñar la pantalla mostrada durante los procesos de carga de niveles o recursos, incorporando elementos visuales coherentes con la temática del juego y posibles indicadores de progreso que reduzcan la percepción de espera.	3.1.5
A121	Implementar consejos de juego	Desarrollar un sistema que muestre mensajes informativos o sugerencias durante pantallas de carga o momentos específicos del juego, facilitando el aprendizaje progresivo de mecánicas y estrategias.	3.1.5
A122	Diseñar pantalla de créditos	Crear el diseño visual de la pantalla de créditos incluyendo estructura de textos, jerarquía de información y coherencia estética con la identidad gráfica del proyecto.	3.1.6
A123	Programar lógica de la pantalla de créditos	Implementar la funcionalidad que gestione la visualización automática o manual de los créditos, incluyendo desplazamiento de texto, animaciones y retorno al menú principal tras su finalización.	3.1.6
A124	Implementar lógica de guardado	Desarrollar el sistema encargado de almacenar el progreso del jugador incluyendo nivel actual, estadísticas, objetos clave y configuraciones relevantes, asegurando persistencia entre sesiones mediante almacenamiento local o remoto y garantizando consistencia de los datos ante cierres inesperados.	3.2.1
A125	Implementar indicador visual	Programar un elemento de interfaz que informe al jugador cuando se esté realizando un proceso de guardado, mostrando una animación o icono temporal que confirme la correcta ejecución de la operación sin interrumpir la experiencia de juego.	3.2.1
A126	Implementar mecanismo anti-corrupcion	Diseñar un sistema de verificación de integridad de datos que permita detectar posibles corrupciones en archivos de guardado mediante validaciones, copias de seguridad o checksums, evitando pérdidas de progreso y mejorando la robustez del sistema.	3.2.1

A127	Implementar lógica de checkpoints	Diseñar e implementar los puntos de control dentro de los niveles donde el progreso del jugador se actualiza. Incluye restaurar la salud de Iris al interactuar con un checkpoint y establecer ese punto como el nuevo lugar de reaparición en caso de derrota.	3.2.2
A128	Implementar guardado y carga de habilidades	Desarrollar la lógica que garantiza que las habilidades desbloqueadas por Iris se mantengan activas de forma permanente tras ser adquiridas, asegurando que, al reiniciar el juego o reaparecer en un checkpoint, las habilidades sigan disponibles sin necesidad de repetir los eventos de obtención.	3.2.3
A129	Implementar sistema de escritura y lectura de datos	Desarrollar el sistema integral de gestión de datos que permite que el estado del juego se mantenga entre diferentes sesiones. Abarca tanto la escritura de datos como la lectura de los mismos.	3.2.4
A130	Implementar captura de métricas	Implementar el sistema de recolección de datos en segundo plano durante la partida. Debe capturar y almacenar variables clave para el análisis, incluyendo el tiempo total por nivel, el daño infligido y recibido, el número de muertes y la habilidad más utilizada por el jugador.	3.3.1
A131	Implementar introducción de nickname	Desarrollar la interfaz y lógica que permita al jugador introducir un nombre identificativo, validando formato y longitud, y asociándolo correctamente a su perfil para uso en clasificaciones o sistemas online.	3.3.2
A132	Implementar generación de UUID	Programar la creación automática de un identificador único universal para cada jugador o sesión, garantizando unicidad en el sistema cliente-servidor y evitando conflictos en el registro de métricas o puntuaciones.	3.3.2
A133	Configurar comunicación cliente-servidor	Establecer la conexión entre el cliente del juego y el servidor remoto mediante protocolos adecuados, gestionando envío y recepción de datos, control de errores de conexión y medidas básicas de seguridad en la transmisión.	3.3.3
A134	Implementar envío de métricas	Desarrollar el sistema que recopile datos relevantes de la partida (tiempo, puntuación, derrotas, uso de habilidades) y los envíe al servidor para su almacenamiento y posterior análisis o visualización en rankings.	3.3.3

A135	Diseñar interfaz de clasificación	Crear el diseño visual de la pantalla de ranking definiendo disposición de nombres, puntuaciones y métricas asociadas, asegurando claridad en la lectura y coherencia estética con el resto de la interfaz del juego.	3.3.4
A136	Implementar listas ordenadas según métricas	Programar la lógica que recupere datos del servidor y los ordene dinámicamente en función de criterios como puntuación, tiempo o nivel alcanzado, mostrando clasificaciones actualizadas y correctamente estructuradas.	3.3.4
A137	Integrar tema musical del tutorial	Incorporar el archivo de audio correspondiente al tutorial configurando su reproducción en bucle, volumen inicial y transición adecuada al abandonar el nivel.	3.4.1
A138	Integrar tema musical del nivel 1	Incorporar el archivo de audio correspondiente al nivel 1 configurando su reproducción en bucle, volumen inicial y transición adecuada al abandonar el nivel.	3.4.1
A139	Integrar tema musical del nivel 2	Incorporar el archivo de audio correspondiente al nivel 2 configurando su reproducción en bucle, volumen inicial y transición adecuada al abandonar el nivel.	3.4.1
A140	Integrar tema musical del nivel 3	Incorporar el archivo de audio correspondiente al nivel 3 configurando su reproducción en bucle, volumen inicial y transición adecuada al abandonar el nivel.	3.4.1
A141	Integrar tema musical del nivel final	Incorporar el archivo de audio correspondiente al nivel final configurando su reproducción en bucle, volumen inicial y transición adecuada al abandonar el nivel.	3.4.1
A142	Integrar tema musical de Umbra	Configurar la reproducción del tema específico asociado al combate contra Umbra, asegurando activación automática al inicio del enfrentamiento y transición adecuada tras su derrota.	3.4.2
A143	Integrar tema musical del Guardián de Hielo	Configurar la reproducción del tema específico asociado al combate contra el Guardián de Hielo, asegurando activación automática al inicio del enfrentamiento y transición adecuada tras su derrota.	3.4.2
A144	Integrar tema musical del Coloso de Ceniza	Configurar la reproducción del tema específico asociado al combate contra el Coloso de Ceniza, asegurando activación automática al inicio del enfrentamiento y transición adecuada tras su derrota.	3.4.2

A145	Integrar tema musical de la Tempestad Dorada	Configurar la reproducción del tema específico asociado al combate contra la Tempestad Dorada, asegurando activación automática al inicio del enfrentamiento y transición adecuada tras su derrota.	3.4.2
A146	Integrar tema musical del Vacío	Configurar la reproducción del tema específico asociado al combate contra el Vacío, asegurando activación automática al inicio del enfrentamiento y transición adecuada tras su derrota.	3.4.2
A147	Implementar tema musical de inicio	Configurar la música de la pantalla principal estableciendo reproducción automática al iniciar el juego y control de volumen vinculado a los ajustes generales.	3.4.3
A148	Implementar tema musical de pausa	Desarrollar la lógica que active una música específica o modifique la actual al pausar la partida, reforzando la diferenciación entre estado activo y estado detenido.	3.4.3
A149	Implementar tema musical de créditos	Integrar el tema musical asociado a la pantalla de créditos configurando su reproducción completa o en bucle hasta la finalización de la secuencia.	3.4.3
A150	Implementar efectos de sonido de movimiento	Incorporar sonidos asociados a acciones básicas como caminar, saltar o aterrizar, sincronizándolos con animaciones y eventos físicos.	3.4.4
A151	Implementar efectos de sonido de combate	Integrar efectos sonoros vinculados a ataques, impactos, daño recibido y derrotas de enemigos, asegurando claridad auditiva y correcta priorización en situaciones con múltiples sonidos simultáneos.	3.4.4
A152	Implementar efectos de sonido de entorno	Desarrollar sonidos ambientales como viento, crujidos o elementos interactivos del escenario, configurando su activación por proximidad o eventos específicos para reforzar la inmersión.	3.4.4
A153	Implementar efectos de sonido de menús	Incorporar sonidos de navegación y confirmación en los distintos menús del juego, garantizando una respuesta auditiva inmediata ante la interacción del usuario.	3.4.4
A154	Programar buses de audio	Desarrollar una arquitectura de audio basada en buses que permita agrupar distintas categorías sonoras (música, efectos, ambiente, interfaz), facilitando el control	3.4.5

		independiente de volumen y la aplicación de efectos globales como filtros o atenuaciones dinámicas.	
A155	Vincular controles de audio	Conectar los controles de volumen definidos en el menú de ajustes con los buses de audio correspondientes, garantizando que las modificaciones realizadas por el usuario se apliquen en tiempo real sin necesidad de reiniciar la aplicación.	3.4.5
A156	Implementar persistencia de volumen	Desarrollar el sistema que almacene las preferencias de volumen del usuario y las recupere automáticamente al iniciar el juego, asegurando coherencia entre sesiones y correcta aplicación de los valores guardados.	3.4.5
A157	Definir estrategia de QA	Establecer el enfoque metodológico para las pruebas del videojuego, definiendo tipos de test (funcionales, de rendimiento y de jugabilidad), criterios de aceptación y planificación temporal para garantizar la detección temprana de errores.	4.1.1
A158	Establecer casos de uso a probar	Identificar y documentar escenarios específicos de interacción del jugador que deban ser verificados, incluyendo flujos completos de navegación, combate, guardado y progresión, asegurando cobertura adecuada de funcionalidades críticas.	4.1.1
A159	Probar navegación de menús	Ejecutar pruebas funcionales sobre todos los menús del sistema verificando transiciones, botones, retorno a pantallas anteriores y ausencia de bloqueos o errores de estado.	4.1.2
A160	Probar persistencia de guardado	Validar el correcto funcionamiento del sistema de guardado comprobando que los datos almacenados se recuperen íntegramente tras reinicios, cierres inesperados o cambios de escena.	4.1.2
A161	Probar carga de escenas	Comprobar la correcta transición entre niveles y pantallas, verificando tiempos de carga, liberación de recursos previos y ausencia de errores visuales o lógicos durante el cambio de escena.	4.1.2
A162	Probar mecánicas principales	Evaluar el correcto funcionamiento de las mecánicas fundamentales del juego, como movimiento, combate, habilidades y progresión, asegurando estabilidad técnica y coherencia jugable.	4.1.2

A163	Realizar sesiones de *playtesting*	Organizar pruebas con usuarios externos o internos para recopilar feedback sobre dificultad, claridad de objetivos y experiencia general, utilizando los resultados para detectar áreas de mejora.	4.1.3
A164	Ajustar curva de dificultad	Modificar parámetros como vida de enemigos, frecuencia de aparición o daño infligido en función de los resultados de testing, buscando una progresión equilibrada y satisfactoria para el jugador.	4.1.3
A165	Ajustar respuesta de controles	Refinar la sensibilidad, tiempos de respuesta y fluidez del control del personaje para mejorar la precisión y sensación de control durante el juego.	4.1.3
A166	Analizar el rendimiento del juego	Evaluar métricas como uso de CPU, memoria y tasa de frames por segundo, identificando cuellos de botella que puedan afectar la experiencia del usuario.	4.1.4
A167	Optimizar scripts	Revisar y mejorar el código del juego reduciendo operaciones innecesarias, optimizando estructuras de control y asegurando una ejecución más eficiente.	4.1.4
A168	Optimizar recursos gráficos	Reducir el peso y complejidad de sprites, animaciones y efectos visuales mediante técnicas de compresión o simplificación, manteniendo calidad visual sin comprometer rendimiento.	4.1.4
A169	Compilar versión jugable para Windows	Generar una build ejecutable del juego adaptada al sistema operativo Windows, verificando que todos los recursos y dependencias se incluyan correctamente.	5.1.1
A170	Verificar recursos incluidos	Comprobar que todos los archivos necesarios (audio, gráficos, scripts y configuraciones) estén correctamente empaquetados en la versión final sin elementos faltantes o redundantes.	5.1.1
A171	Crear instalador	Desarrollar un instalador que facilite la distribución del juego, configurando rutas de instalación, accesos directos y desinstalación limpia del sistema.	5.1.1
A172	Redacción de informe de cierre	Elaborar un documento final que recoja resultados del proyecto, cumplimiento de objetivos, análisis de riesgos gestionados y conclusiones generales del desarrollo.	5.2.1

A173	Redacción de la memoria	Redactar la memoria completa del proyecto incluyendo contexto, objetivos, planificación, desarrollo técnico, pruebas realizadas, resultados obtenidos y reflexiones finales, estructurándola conforme a los requisitos académicos establecidos.	5.2.1
A174	Realizar manual de usuario para desarrolladores y usuarios finales	Elaborar los manuales destinados a los usuarios finales y/o desarrolladores, explicando cómo instalar, configurar y utilizar el videojuego, así como las funcionalidades principales y las recomendaciones de uso.	5.2.2